

# Immigrant Collection Action ML Project

## Memo

2025-12-01

First I give a very high-level overview of both the problem and the solution. In the second section, I discuss the parameters of the problem in more depth, starting with conceptual issues, then structural questions, and finally more technical methodological challenges. Then, in the third section, I lay out the potential solutions in more detail, explaining what I'm currently doing and why (and why I'm not taking alternative approaches). This includes a more detailed explanation of the methods I'm using and the particular issues they address. Section four details what I've actually done so far and my results. Finally, I conclude with a light sketch of where we're going next (at a high level) and a more detailed discussion of immediate next steps/questions (i.e., likely issues for the meeting).

## Overview

### The Problem

As I understand it, our overarching goal is to better understand the history of contentious collective action by (or for and/or against?) immigrants in the United States. Our work in Voss, Lauterwasser, and Bloemraad (2024) (and in the ongoing domains of contention paper) begins to do this by looking at the period from 1960-1995, telling the “prehistory” of the modern movement. This demonstrated the potential of a population at risk approach (cf. McAdam and Boudet 2012) for uncovering and telling fuller histories (or “prehistories”) of contentious collective action in the absence of well-defined or identified movements. However, this approach is founded on protest event analysis (considering CCA at the event level) (see Hutter 2014) and thus requires event level data. Moreover, it also requires that data to be, if not complete (which no dataset will ever be), at least relatively well-defined in its completeness.<sup>1</sup>

---

<sup>1</sup>While no event dataset is ever likely to be actually complete, we need to have some kind of handle on the nature of its incompleteness and that incompleteness should reasonably match the goals of the investigation. For instance, while the *New York Times* is of course incomplete in its reporting, we can relate that incompleteness

This leaves us with a problem: while the *Dynamics of Collective Action* (DoCA) dataset covers the period from 1960-1995 in relatively convincing fashion, there is no real comparable dataset covering either the thirty years following or the nearly two hundred years preceding it. Constructing such datasets is, of course, *extremely* resource intensive using traditional methods, requiring huge numbers of person-hours and teams of well-trained coders (Oliver et al. 2023). We currently lack the resources to do this (and even if we had them, it is not immediately clear that this would be the best use for them). We do, however, have access to the raw data we would need (i.e., the *NYT*), with some caveats (see below on why we’re starting with headline classification). What we want, then, is a way to go from the raw data to a DoCA-like (or Voss et al. (2024) ICA dataset-like) dataset, without spending thousands of hours and/or dollars.

Machine learning may offer a solution.

## The Solution

Machine learning (ML), especially text analysis and classification, has been used in the social sciences (including sociology) for more than a decade (see, e.g., Grimmer and Stewart 2013). What the machine “learns” in ML is a way of representing the input data (Chollet 2025). ML is exciting for text data because data representation is *the* problem in quantitative text analysis: e.g., how do we represent the text of an email such that we can judge whether it’s spam or not?

An ML model thus consists of two things: a way to transform the input data (i.e., to represent it) and a way to evaluate the quality of those representations (generally called a loss function, an objective function, the loss). In supervised machine learning, the quality of the generated representations is measured against a set of labels (e.g., a headline mentions CCA or not; an image contains a cat, a dog, or neither; etc.), while in unsupervised learning, quality is defined with reference to some relative, internally defined criterion (e.g., a clustering algorithm might aim to maximize the average distance between clusters). Ideally, this feedback signal also provides some guidance for how to adjust the transformation to (hopefully) improve the quality of the resulting representation.

We can imagine fitting a single linear regression like this: start with a random coefficient and intercept, get the average error over the input dataset, adjust the coefficient and intercept (upwards if the average error is positive, downward if it is negative), and repeat. Obviously this is not a particularly efficient way to fit a linear regression, but all machine learning models are trained according to the same basic loop.

What we want, at a high level, is an ML model that takes the text of a newspaper article and transform it such that the representation that comes out tells us whether it reports an ICA

---

to some understanding of “newsworthiness” (Earl et al. 2004; Oliver, Hanna, and Lim 2023) and say that a dataset based on the *NYT* will be better suited to investigating national level trends than a dataset drawing on a local paper.

event, how many people were involved, etc. Whether we can create such a model (or models, see below for a discussion of splitting up the problem) will depend on the input data we have available, the possible ways we can transform that data, and the nature of the feedback signal from output to transformation.

## Deep Learning

This would be an intractable problem if we were limited to simple linear regression as our transformation. Fortunately, we are not. I will get into the details later, but for the moment I want to give the very broad strokes of how the *models* we're using and the *processes* for training them differ from, e.g., the linear regression example above.

*The models:* A linear regression is limited in two ways that are relevant here. First, it is, of course, linear. And second, it only performs a single transformation on the data (i.e., it goes from input to output in a single step). We will be considering methods with neither of these limits. Specifically, deep learning is a subset of ML in which a model performs multiple transformations in sequence. A deep learning model is a stack of “layers” of transformations, with each layer defined by its own set of parameters. Each individual layer may be a linear transformation, but generally step in each layer is to pass the output through some kind of non-linear function. The result is that deep learning models are extremely flexible.

*The process:* In the linear regression example, the model has just two parameters (the regression coefficient and the intercept) and the input dataset is likely relatively small (consisting of a single input value and single output label per data point). But a deep learning model may have millions or billions of parameters. As such, adjusting all the parameters in the same way based on the direction of the error would be completely useless. Likewise, only updating after processing the entire dataset would be extremely computationally wasteful. So, instead, we train deep neural networks in batches using gradient descent. Training in batches just means that we update the model's parameters after every  $n$  data points, where  $n$  is the batch size, rather than only after processing the entire dataset.

Gradient descent makes the feedback from model output to updated parameters dramatically more helpful by using loss functions (the function we use to evaluate the quality of the model's output) that are *differentiable* with respect to the parameters of the model. Thus, after every batch of outputs, we take the partial derivative of the loss with respect to each parameter and tweak them all in the “downhill” direction (i.e., the direction that would lead to a lower loss).

## Parameterizing the Problem

Previous attempts to use machine learning to identify and code protest events in text (newspaper) data have, by our metrics, failed (Hanna 2017; Oliver et al. 2022; Oliver et al. 2023).<sup>2</sup> Our main goal is to leverage ML to reduce and/or eliminate the intense resource requirement of by-hand protest event identification and coding. The ongoing work by Pam Oliver, Chaeyoon Lim, and Alex Hanna has had substantial success in creating effective workflows (which may include ML based assistance, although this is not always clear) for by-hand coding and done very impressive conceptual work in understanding some core problems in protest event analysis; however, it has not resulted in substantial success in automated identification and coding.

## Conceptual

But the conceptual work done by Oliver, Hanna, and Lim (2023) is extremely helpful in understanding some of the challenges we face. These can be summarized in two broad points: 1) the complexity of CCA/ICA as a phenomenon; and, 2) the difficulties of the *NYT* (and newspapers generally) as a source.

If we want to have a model that can differentiate articles that include contentious collective action events from those that do not, then we need a rigorous definition of what CCA is. This definition does not necessarily need to be conceptually perfect, but it does need to be clearly prototyped in the data. That is to say, there need to be enough labeled exemplars that the model can inductively learn the difference between articles that do and do not match the definition. This is difficult in the obvious ways; as Hanna (2017) notes, “the social object of the ‘protest’ is itself heterogeneous, difficult to define, and requires explicit boundaries to separate it from routine crime, sport hooliganism, terrorism, or other forms of political violence” (2017:11–12). But it is also an issue in slightly less obvious ways as well: even if *we* have a clear conceptual definition distinguishing, e.g., CCA and terrorism, we cannot teach the *model* that definition if we lack the exemplars to do so. And we might lack exemplars in two distinct ways: first, events we would label terrorism could be labeled as CCA in the training data; and, second, we could have too few exemplars on one or both sides of the decision boundary.

The good news is that we retain a decent amount of flexibility, due to the level of detail in DoCA’s coding. Despite relying on DoCA as a major source of training data, we are not limited to replicating their definition, so long as the distinctions we want to make are things that are coded for in DoCA and/or in the *NYT* indexing service descriptors in the NYT Annotated Corpus (more about both of these below).

---

<sup>2</sup>I should note that I have not done an in-depth search for other attempts (at least not recently), so it’s possible we’re less ahead of the curve than it sounds like here.

Oliver et al. (2023) push this definitional issue even further, arguing for the “inherent ambiguity and relationality of protest as a concept” (2023:21). This brings us to the difficulties of the *NYT* specifically, and newspapers generally, as a source. They go beyond existing discussions of selection bias and basic issues of “newsworthiness” (see, e.g., Earl et al. 2004) by focusing on the report-event dyad, rather than events or reports per se. This facilitates a focus on verifiability and de-duplication grounded in a multi-step, iterative, coding process: multi-step and iterative coding process: gather articles mentioning events -> identify events within those articles (including annotating relevant text) -> “event adjudication” in which events are matched up across articles and *all* annotated text is explicitly accounted for (even if via “this should not be included because xyz”), resulting in “authorized events” in the table of events -> article-event links created. And they are able to show that it matters: of 2682 article-event pairs, only 7% involved one article per event and one event per article. And, while 69% of events were mentioned only once, the 16% of events mentioned in three or more articles made up fully half of the news mentions of events.

Their conceptual framework and empirical findings may not be an immediate concern, but I think we will need to take them seriously as we move forward.

## Structural

As has perhaps already been made clear, I think it is helpful to understand this overarching problem of ICA identification and coding as in fact being composed of multiple, only somewhat related, tasks. Hanna (2017) divides the problem into three distinct tasks: the “haystack” task, the closed-ended coding task, and the open-ended coding task. The haystack task could also be called the “identification” task; that is, the task of accurately identifying the relatively small subset of articles (the needle) in the many thousands for a given newspaper corpus (the haystack) that report a contentious collective action event at all. The closed-ended coding task is a classification task: based on the contents of the article, assign it to one of a discrete set of possible buckets (form, target, and issue are the three variables she considers as closed-ended coding tasks). Open-ended coding, on the other hand, involves extracting relevant information that could take on an indeterminate set of possible values (e.g., the number of participants, the names of SMOs involved).

For us, the haystack task is more complicated, since our ultimate goal is to identify the even smaller subset of articles reporting ICA events. But I believe it likewise useful to break this problem down into its constituent parts: identifying reports of CCA events and reports involving immigrants, before using that information to help identify the overlap that is reports of ICA events. This approach is grounded in the exemplar-based, inductive nature of ML: we have far more examples of CCA articles and immigrant articles separately than we have of articles reporting ICA events. By first treating these problems separately, we can dramatically narrow the task of identifying ICA events.

Additionally, I want to point to an alternative conceptualization of the two coding tasks: rather than closed- versus open-ended, we can think about the extraction of specific, concrete information (e.g., number of participants, date, location, organizations involved) versus the coding of more abstract or conceptual characteristics (e.g., domains of contention, the kind of claim being made, etc.). This distinction is relevant because these problems may be amenable to different strategies. In particular, we might be able to make considerable headway in the coding of conceptual characteristics using unsupervised clustering methods, even absent the training data necessary for the model to learn our predefined categories (Grootendorst 2022). We can imagine, for instance, that word embeddings and concept mover’s distance (cf. Stoltz and Taylor 2019, 2021) might allow us to reproduce clusters of events corresponding to our domains of contention. Likewise, while we are unlikely to find large volumes of existing training data for, e.g., coding domains of contention, we might have an easier time repurposing alternative sources of data to help train a model to extract the number of participants in an event report (Wankmüller 2022).

Finally, we might think about a fourth task, intermediate between the haystack tasks and everything else: the parsing the text of an article to identify the text related to each of the one or more events reported therein. Effectively performing this task both makes the coding tasks dramatically easier and would enable us to meaningfully move from talking about “articles” to “events”, which, as Oliver et al. (2023) argue, a lot of protest event analysis fails to do.

## Methodological

At a very high level, the methodological challenge is in finding a way to turn human readable text into meaningful quantitative representations that the computer can do something with and learn things from. But this is the general challenge of computational text analysis, and we do have promising ways to do this, discussed in more detail below (Devlin et al. 2019; Stoltz and Taylor 2019; Vaswani et al. 2017). Instead, what I want to do here is identify and discuss a few of the more specific challenges in the nature of our goal.

First, we have a data problem: while we actually have a pretty solid number of labeled positives to use for training (for both CCA events and articles relating to immigrants, separately), we have zero labeled negatives. As a result, instead of a standard Positive-Negative (PN) learning problem, we have a so-called Positive-Unlabeled (PU) learning task on our hands (Bekker and Davis 2020). This complicates things somewhat, since the whole training process is premised on being able to show the model labeled examples from mutually exclusive classes. If we were to just assume that all unlabeled articles were negative (i.e., not reporting CCA events), we would get a biased (and poor) classifier, in proportion to level of unlabeled positives.<sup>3</sup>

---

<sup>3</sup>Alternatively, if the labeling mechanism is non-random (i.e., there’s a reason the labeled articles were labeled and the unlabeled ones weren’t) we could end up with a very good, unbiased, classifier *that identifies whether an article would be labeled or not*, which is obviously a different thing.

I'm confident that we need to treat this as a PU problem based largely on findings from Hanna (2017), who found that a classifier trained on the DoCA data turned up significant numbers of false positives when applied back to the source *New York Times* data. However, when she hand coded a subset of these articles, she found that 31 of the 50 she checked were “false false positive”; i.e., articles that should have been in DoCA but were not (Hanna 2017). This is worrying news for DoCA itself (the classifier was *correctly* identifying articles that had been omitted by DoCA coders, for whatever reason), but is actually somewhat encouraging for our project, in two ways. First, it demonstrates that, even within the span already covered by DoCA, there is more contentious collective action (or at least articles about it) to uncover. Second, and more helpfully, it shows that a classifier trained on DoCA articles is capable of correctly generalizing and identifying new events that were not in the original training data. And this was in 2017, using far weaker methods for representing and learning from text data than we have access to now (i.e., term frequency/inverse document frequency rather than word embeddings and support vector machines, logistic regression, or other linear methods rather than neural networks). (Although, to be fair, I believe Hanna may have been using the whole article.)

Second, identifying ICA events is an extremely imbalanced problem (as indicated by labeling it the “haystack” problem in the previous section). This complicates the training process: the vast majority of articles are easy negatives (i.e., they're obviously not reporting on CCA events). As such, they don't teach the model very much, but there are so many of them that they can overwhelm the useful signal provided by hard negatives (i.e., cases where the model is uncertain) and positives.

Third, even if all we want to do is expand the DoCA/ICA dataset forward (i.e., from 1995 through the present, or at least 2007), historical variability will start to become a problem. Obviously there is linguistic variability: what words mean changes over time and so the words used to describe a protest in 1960 may be different than those used in 2025. More than that, however, not only the meanings of individual words, but the larger frames, attitudes, and assumptions that shape how events are reported (and what even counts as newsworthy) also change over time. This creates both very concrete problems (e.g., if the size of a protest was seen as less important in a given period, then reports may simply lack that information) and more subtle ones (e.g., if militant protest is seen very negatively, it may become more difficult to differentiate it from other negatively valenced contentious acts, such as terrorism). These are different kinds of problem, which therefore have different kinds of solution (see the discussion in the next section).

## What We're Doing and Why

I have largely focused on the “haystack” problem (i.e., identifying articles reporting on ICA) thus far and will therefore be focusing on it here. I'll discuss some thoughts on approaching the other coding tasks at the end. Turning to the haystack problem, I'm going to start by giving

a very high-level summary of what I’m planning to do, then elaborate on each individual piece in subsections below. In the next full section, I’ll detail exactly what’s done right now.

I am using a pre-trained transformer based model (specifically RoBERTa) as the backbone for a multi-headed classifier which takes as input the headline and lead sentence of an *NYT* article and returns the probabilities that the article reports 1) a CCA event, 2) on immigrants, 3) on events in the US, and finally an overall probability that the article reports an ICA event taking place in the US. The training data come from a combination of DoCA and the NYT Annotated Corpus.

Before training the classifier, the RoBERTa backbone was given a round of domain-adapted pretraining on headline-lead pairs (a similar strategy may be employed to deal with linguistic variability). The CCA and immigrant identification tasks are both clear Positive Unlabeled learning tasks and, to different degrees, highly class imbalanced. As such, those classification heads are trained on re-balanced batches with a focal loss and virtual adversarial training. The US/not US identification task is far more balanced and far simpler. The overall final classifier output combines the output of the individual task heads with a combined ICA/not head.

Once the structure of the model is fully setup, I’ll run a search across potential hyperparameter values in order to find the ideal set for actually training and using the model. Then we can fully train the model and evaluate its performance on a held-out test set, ideally one which has been hand coded to be fully and correctly labeled (i.e., PN rather PU). There are a number of different metrics we can use to evaluate performance and decide what counts as a “good enough” model, depending on how exactly we want to use it. At this point we will also calibrate the model and our decision threshold, based in part on these same metrics and our intended use for the model.

## Pre-Trained Transformer Based Model

This is the reason that what we’re trying to do is at all remotely feasible. In essence, the pre-trained transformer backbone is what “solves” the methodological problem of going from human readable text to meaningful quantitative representations. This is, as previously discussed, not a straightforward problem and getting to the transformer solution takes a few steps. We could imagine, for instance, assigning each word in the dictionary a number, but that number would tell us nothing beyond where in the alphabet the word was located. We could represent words in terms of their frequency, and this can even be relatively useful for document classification tasks (e.g., predicting the author of a document based on idiosyncratic word choice). However, this still tells us little to nothing about what the word being represented means.

The first major step towards what we’re doing here is the innovation of word embeddings. Word embeddings are numerical representations of words (called word vectors) that capture their meanings based on the contexts in which words appear across large text corpora (Mikolov et al. 2013). The basic insight at the core of the translation is that words are defined by their



use and thus by the words around them. The result is that words that are used in similar ways will have similar vector representations and will thus be literally close to each other in the embedding space. The upshot of this is that spatial relationships in the embedding space are meaningful, with semantically and/or conceptually similar words closer together and dissimilar words farther apart.

The problem with word embeddings as first conceived is, basically, polysemy: a single word can have multiple (quite different) meanings depending on its context. The word “bank”, for instance, appears in (at least) two very different sets of contexts. In one, “bank” is related to money and accounting, in the other, it’s related to rivers and dirt. But basic static word embeddings can’t differentiate these contexts, so the numeric representation of “bank” should be close to both “money” and “river”. Ideally, what we would want is a way to still have an embedding space that’s meaningful, but where the numeric representation a word receives can vary based on its exact context.

Transformers get us this.

In short, rather than creating a “static” word embedding where a word’s numeric representation (its position in the embedding space) is, in some sense, its average across all uses in the training corpus, transformers create an *encoder* that dynamically generates a numeric representation for each word in a given sequence based on the words in that sequence. This is made possible, in part, by a mechanism called “attention”, which allows the representation for each word to be influenced variably by all the other words in the sequence (Vaswani et al. 2017). Thus, instead of relying on some kind of built-in heuristic like “closer words matter more” the model is actually able to learn the relevant dependencies across a sequence.

It turns out that a model structured this way can get extremely good at “general English understanding” if trained on a sufficient amount of data with an appropriate task. The specific model we’re using, RoBERTa (Liu et al. 2019), was trained using “masked language modeling”: the model is fed a series of sequences (English sentences from a huge corpus of documents) in which a small percentage of words have been “masked” and replaced with another token. The model must then predict the what the masked word originally was. This is a “self-supervised” task, in which the labels used for training are generated from unlabeled data.

Training such a model is incredibly computationally expensive and requires huge amounts of digitized text. Thus, we are using a *pre-trained* version of RoBERTa as the *backbone* in our modeling. In other words, we’re taking a version of RoBERTa that has already been trained using masked language modeling on a huge amount of English text, and we’re using it to encode our text (i.e., create these meaningful word embedding representations). We then take those representations and feed them into a smaller model built on top of RoBERTa.

## Domain-Adaptive Pretraining

While fully training a transformer language model from scratch is prohibitively expensive, we can tweak that pretraining a bit in order to make the model more useful for our specific case. RoBERTa was trained on a large and relatively diverse corpus including English Wikipedia, books, news, and web content (Liu et al. 2019), but we’re going to be tasking it with understanding (so to speak) newspaper headlines and articles. This is a specific problem “domain” that overlaps with general English understanding but will almost certainly have its own unique features.

So, engage in domain-adaptive pretraining (Gururangan et al. 2020): before building our actual models of interest (e.g., for identifying ICA events), we first train just the RoBERTa backbone with the same masked language modeling task as was used in its original training. However, instead of using a generic English corpus, we use our own corpus of newspaper data. This has been found to lead to significant improvements in downstream tasks, especially for domains that are more idiosyncratic (Gururangan et al. 2020).

This same basic idea has two additional potential applications. First, it shows up in a more narrow way in task-oriented transfer learning (also called intermediate task transfer). Here, rather than adapting the model’s broad understanding of English to the more specific “domain” of *NYT* style English, we are trying to give it a head start on learning our specific classification task (i.e., identifying articles reporting ICA events) by training it on “similarly shaped” task first (Phang, Févry, and Bowman 2019; Pruksachatkun et al. 2020; Vu et al. 2020). The idea is to try and overcome the data limitations of the target task (that is, that we don’t have a *ton* of labeled data and we don’t know for certain that all positives are labeled, more about which below) by pretraining on a task for which lots of well labeled data is available (Vu et al. 2020). For instance, we could train the model to identify which section of the paper an article was in, or whether it has a given indexer-assigned label. To anthropomorphize a little bit: base RoBERTa has only ever been taught to encode text, it’s never been asked to make judgments about (i.e., classify) it. Without this step, we’re throwing RoBERTa into the deep end with a very tricky classification task and no prior experience. But if we first train it be very good at, e.g., identifying which articles were in the business section, then it at least has experience making judgments at all, and just needs to adapt that framework to a new application.

Second, a version of this may be what allows us to expand our model from just the past few decades to the full *NYT* corpus. In short, in the same way that we can use DAPT to teach base RoBERTa the idiosyncracies of *NYT* text, we can use text from 1900 to teach base RoBERTa about the idiosyncracies of English in 1900. For this to be useful, we need to do it in a way that teaches the model to better understand text from 1900 without causing it misunderstand text from the year 2000. The most basic way to do this is by bundling time (date) information together with the text during training and inference (Dhingra et al. 2022; Rosin, Guy, and Radinsky 2022). The standard transformer language model (like RoBERTa) is pretrained using only raw text, but there’s no reason that other pieces of data can’t be incorporated as well. So we can simply prepend a special time token (e.g., “<1900>”) to

the text being fed to the model. We can further add a specific “temporal masking” task to training, such that the time token is masked and the model learns to predict it. Because of the attention mechanism (discussed above) this time token will help the model interpret every other token in the sequence: in the same way that the model will encode “bank” differently if it appears in a sentence with “river” versus “teller”, it can learn to encode words differently based on the time token (Rosin et al. 2022). We could even take this further by modifying the attention mechanism itself so that it is structurally/mathematically conditioned on the provided time information (Rosin and Radinsky 2022), though this may be beyond the scope of the current project.

## Data

We are using a combination of the Dynamics of Collective Action dataset (DoCA)<sup>4</sup> and the NYT Annotated Corpus.<sup>5</sup> We further enrich the DoCA data with our own Immigrant Collective Action labeling (Voss et al. 2024). These datasets cover overlapping but different periods: DoCA includes over 23,000 events from the years 1960-1995, while the annotated corpus contains 1.8 million articles covering 1987-2007. Critically, the annotations to the annotated corpus include tags applied to each article by an indexing service. Using these tags dramatically expands our labeled data, getting us 20329 CCA articles, 11329 immigration articles, and 661 ICA articles (i.e., the overlap). This is an average of about 950 CCA, 530 immigration, and 30 ICA articles per year from 1987 to 2007. Obviously, this is still not a ton of ICA articles, but I don’t think this is going to be a problem (see Multi-headed Classifier, next section).

## Headlines and Ledes

You will note that 20329 CCA articles is less than the 23k CCA events in DoCA. That’s because this number currently includes only those events from DoCA that have a corresponding article in the annotated corpus (i.e., happened from 1987-1995). DoCA does provide article headlines, but I’ve decided to use headlines plus lede sentences (which DoCA does not include). I’ve made this choice for two reasons. First, ledes add *so* much content while still being short enough to be computationally straightforward. There are many headlines that are simply too short or simple to provide a real meaningful clue as to whether they match our classification. Adding lede’s solves this problem.

Second, headlines and ledes are just as easy to get as headlines alone: while the NYT doesn’t provide API access to the full text of articles, it does provide API access to *headlines* (and other

---

<sup>4</sup>For details on DoCA, see the website and coding manual: <https://collectiveaction.stanford.edu>

<sup>5</sup>The NYT Annotated Corpus was previously made available by the University of Pennsylvania Linguistic Data Consortium (LDC-NYT), but it is no longer publicly available. Reference information for the corpus is available here: <https://catalog.ldc.upenn.edu/LDC2008T19>

metadata) through their “Archive” API (link: <https://developer.nytimes.com/docs/archive-product/1/overview>). This API will give you the metadata (including the headlines and, usually, ledes) for all the articles in the NYT from a requested month. Given that the NYT allows up to 500 API requests per day (see answer to question 11 of this FAQ: <https://developer.nytimes.com/faq>), it should take a little more than 4 days to create a dataset of every single published headline from 1851 to the present (well, through 2019, the current limit for this API).

That said, I have not yet done this for the DoCA articles from 1960-1984. (See Next Steps, at end.)

## Defining ICA

The set of annotated tags that I’m using as labels for CCA and immigration are currently a first pass. They will need to be both conceptually and empirically validated. (See Next Steps, at end.)

I count the following tags as CCA: “demonstrations and riots”, “hunger strikes”, “picketing”, “civil disobedience”, “strikes”, “boycotts”

And these tags as immigration: “immigration”, “immigration and emigration”, “immigration reform and control act of 1986”, “asylum (political)”, “migrant labor”, “deportation”, “refugees and expatriates”, “illegal aliens”, “visas”, “illegal immigrants”, “immigration and refugees”, “citizenship”, “Border Patrol (US)”, “Customs and Border Protection Agency”, “Citizenship and Immigration Services”, “Customs and Border Protection Bureau”, “Immigration and Customs Enforcement”, “Immigration and Customs Enforcement Agency”

One thing worth noting is that this does catch a somewhat different definition of CCA than DoCA does, both more and less expansive (including strikes + “normal” political rallies, not including lawsuits). There are likely enough DoCA events separately to get the stuff that this misses, but there isn’t necessarily a good way to narrow this down. That said, we may want a different definition from DoCA anyways. Alternatively, we might want to aim for finer grained classification (e.g., street versus boycott versus conventional versus lawsuit).

## Multi-headed Classifier

The main reason that I’m not worried about our small number of specifically labeled ICA articles (as opposed to just articles about CCA events or immigrants) is that I’m planning to break the problem up. As discussed above, the purpose of the transformer model backbone is to create a meaningful quantitative representation of any text we put into it. It does this by embedding each word of that text into a many-dimensional vector space (in our case, this space has 768 dimensions). To go from this meaningful representation to *classification* we need a way to squeeze this complex, multidimensional representation down in a single output

(i.e., yes/no). This is basically fitting a logistic regression with 768 independent variables and the dependent variable is the probability of a “yes” label. But there’s nothing preventing us from fitting *more than one* regression, with a different classification task (and thus different set of target output labels) each time. Put another way, the same underlying 768-dimension representation of the input text can be fed into as many different output layers as we like, which will squeeze down those dimensions into different sets of yes/no predictions. We call each of these outputs a separate classification “head”.

By having multiple classification heads, then, we can break down our actual problem of interest (identifying immigrant collective action events in the US) into separate component parts. Specifically, the plan is to have separate classification heads for: contentious collective action events (general), articles about immigrants, and articles about US events. We can even route the output of these different heads back together by simply making them inputs to another layer. For us, this means we can also have a final combined output head that takes the prediction for each component question (is it CCA? does it involve immigrants? is it in the US?) as inputs, alongside the same 768-dimension representation of the input text. This can allow the model to better understand and identify weird edge cases or dependencies among these components.

Then, when training the model, we calculate the loss for each intermediate output (i.e., each component question) as well as the final output layer. We then use these to update the weights of each output layer accordingly. We can also combine the losses and use the combined loss to update the weights of the backbone model (though the payoff for this can be pretty variable, depending on empirical relationship between our different outputs in the training data).

Ultimately, this design allows us to leverage the larger labeled datasets for general collective action and immigrant involvement to make the end goal immigrant collective action classification task tractable despite the far smaller number of labeled articles. Each head shares the same backbone representations but learns task-specific decision boundaries.

## Positive-Unlabeled Learning and Class Imbalance

As discussed above (in the “Methodological” subsection of “Parameterizing the Problem”), identifying articles reporting ICA events is made especially difficult by two things: 1) the lack of labeled negative examples (i.e., in our training data, we have articles we know report ICA events and articles where we don’t know, but not articles that we know for certain don’t report ICA events); and, 2) severe class imbalance (only a vanishingly small minority of articles report ICA events).<sup>6</sup>

Breaking the problem down into its component parts does not, at least in our case, solve either of these issues, but they do impact the different component tasks differently. Indeed, the US/not US task is largely free from both of these issues: it is quite easy for us to have a

---

<sup>6</sup>These problems are theoretically distinct, but often overlap in practice (as they do here)

confident set of both labeled positives and negatives and for that set to be relatively balanced. However, both the CCA and immigrant involvement identification tasks do suffer from both of these issues.

## Positive-Unlabeled Learning and Its Assumptions

There are essentially three kinds of approaches to positive-unlabeled (PU) learning (Bekker and Davis 2020) which solve the problem of unlabeled data in different ways:

- Two-step: identify reliable negative samples and then proceed with normal positive-negative (PN) learning using those negative samples. The obvious problem is that you need some way to identify those negative samples. The slightly less obvious problem is that your classifier may perform worse on the test set (or on real data) if your method for identifying negatives doesn't effectively cover the full distribution of negatives. That is, even if none of your identified negatives are actually positives, you still need to be doing a good job of identifying all the different kinds of negatives, or the model won't be able to learn about them.
- Biased learning: here, we treat the unlabeled data as noisy negative samples and then use some method to try and adjust for the noise, generally by down-weighting the "negative" samples in some way (e.g., penalizing misclassified positive samples more harshly).
- Class prior: these methods leverage knowledge (or estimates) of the positive *class prior*  $\alpha = \Pr(y = 1)$  (i.e., the probability a random sample is positive). This can be done in different ways (Bekker and Davis 2020), but the one which concerns us is *unbiased* PU learning (du Plessis, Niu, and Sugiyama 2014; Elkan and Noto 2008). Here, the class prior allows us to treat unlabeled data as both weighted positive and negative data at the same time (Kiryo et al. 2017).

All methods for PU learning rely on making some assumptions about the labeling mechanism, the data, or the class prior. I discuss the assumptions most relevant for our case, and omit the rest:

- The labeling mechanisms (i.e., how it comes to be that some positive samples are labeled but others are not):
  - Selected Completely At Random (SCAR): labeled positive samples are a completely random sample from the set of all positives, with all samples having an equal probability of being labeled. Note that this implies that there is no correlation between any of a sample's attributes and its chance of being labeled. This is the strongest assumption and is required for class prior methods (including unbiased PU) as well as some methods in the other two categories.
  - Selected At Random (SAR): labeled positive samples are a random biased sample from the set of all positives, with the bias towards selection being dependent on a sample's attributes. If we understand the labeling mechanism, then we can leverage

our understanding of the labeling propensity to get an unbiased classifier. If we don't, then we need additional assumptions.

- Probabilistic Gap PU (PGPU): a special case of SAR in which the chance to be labeled is proportional to the “probabilistic gap”, the difference between the sample’s probability to be positive, given its attributes, and its probability to be negative, given its attributes. In other words, positive samples that are more similar to negative samples are less likely to be labeled.
- The data:
  - Smoothness: samples that are similar to each other are more likely to have the same label. If this assumption holds, then samples that are far from all the labeled (positive) samples should reliably be negative.

These assumptions are, I think, the ones that depend most directly on our domain expertise: how *do* we think the labeling mechanism worked for our different sets of labels? Do we think that negatives are necessarily very different from positives?

Currently, I am proceeding as though the labeled positive samples were selected completely at random. I doubt that this is literally perfectly true, but I think it’s plausibly mostly true. However, I am absolutely open to disagreement here. It also may depend, to some degree, on the exact definitions of contentious collective action and immigrant involvement that we want to use.

That said, assuming the labeled samples were SCAR, I am using a modified version of unbiased PU learning (Ji et al. 2023; Kiryo et al. 2017) and testing different methods for estimating the class prior (Garg et al. 2021; Ivanov 2020). Crucially, there are specific two-step (and potentially biased) approaches that should be workable, assuming we’re not comfortable with the SCAR assumption. Additionally, the modifications to the unbiased PU approach will be useful regardless of whether we stick with that method, since they also help address class imbalance and are applications of more general techniques to unbiased PU, rather than specific innovations on it.

## **Class Imbalance and Other Techniques**

Class imbalance simply refers to the fact that, even ignoring the fact that not all positives are labeled, there are far far more negative than positive samples. That is, there are far more news articles that do not report a CCA event (or immigrant involvement) than there are articles that do. This is even more of an issue for class prior PU learning, since the class prior is used to weight the loss (meaning that the already uncommon positive samples will contribute even less to the training, relatively speaking).

We attempt to address this in three ways:

- Re-balanced training batches: instead of each training batch consisting of a random draw of samples, we instead draw from positive and unlabeled samples separately (currently one positive for every nine unlabeled). This ensures that every batch has some signal from labeled samples in it (Ji et al. 2023).
- Focal loss: a modified version of the cross-entropy loss (extremely common in deep learning) that dynamically weights each sample based on its difficulty: the more confident the model is in a (correct) prediction, the less weight that prediction contributes to the loss (Lin et al. 2020). Functionally, this serves to focus training on the small subset of difficult to classify samples, preventing the training from being overwhelmed by the multitude of easy to classify negatives (or, in our case, unlabeleds) (Ji et al. 2023).
- Virtual Adversarial Training: VAT attempts to ensure distributional smoothness in the output of the model (Miyato et al. 2019). Practically, this is a way of making the model more robust, such that small changes in input don't result in large changes in output. However, it also appears that it can improve the generalizability of the model as well (Liu et al. 2020). Conceptually, VAT works taking the embedding of the input text and perturbing in the direction that leads to the largest divergence in the output and we tweak the model parameters so as to maximize that divergence (as opposed to minimizing the standard loss).

## Hyperparameter Search

After preliminary testing, once we have a model structure that we basically like, we then need to search across potential hyperparameter values in order to find the ideal set for actually training and using the model (Chollet 2025). Hyperparameters are simply parameters that control the learning process of a model. This includes things like the learning rate (how large a step we take each time we tweak the model parameters), the batch size  $n$  (the number of samples that we process at once and after which we update the model), or the scaling factor for the focal loss. Unlike the model's normal parameters, they are not learned from the data directly. Instead, we must test each combination of hyperparameters by using them to fit the model. This can be a laborious process, but some shortcuts are possible (e.g., aborting training runs that are clearly worse than previous tests).

## Evaluation and Use

Once we've fully trained the model, we can then evaluate its performance on a held-out test set. Evaluating the performance of PU learning models is made somewhat more complicated by the fact that the test set is also largely unlabeled. That said, it is still possible, with adjustments and assumptions (Bekker and Davis 2020). Alternatively, we could put in the time to fully label a test set. Such a test set would not need to be terribly large (e.g., not more than 1000), so this may well be worth doing.



There are a number of different metrics we can use to evaluate performance, which we can discuss in detail in future. The important conceptual point to note is the precision/recall tradeoff: precision is the ratio of true positives to all positives (i.e., true positives / (true positives + false positives)), while recall is the fraction of positives the model found (i.e., true positives / (true positives + false negatives)). Generally speaking, there is going to be a tradeoff between how well you avoid false positives (precision) and how well you capture all positives/avoid false negatives (recall).

This tradeoff shows up in model calibration: the model doesn't output binary predictions (yes/no), it outputs *probabilities*; i.e., its confidence that a given input is an ICA event. What we do with those probabilities is up to us. Due to some of the complexities in dealing with PU learning and class imbalance, we can basically guarantee that we don't want to use .5 as a cutoff (i.e., label an article as ICA if the model's confidence is greater than one in two). Ultimately, we'll want to test different decision thresholds using first the test set and then by experimenting with some real new data.

There is not "correct answer" in all of this: how we balance the precision/recall tradeoff, what we consider to be a "good enough" model, etc. are all practical questions whose answers depend on how we are going to use it. To figure out where we come down, we'll want to think about questions like:

- what error rate is acceptable?
- what kind of errors? are some kinds of errors preferable to others?
- is there a level of hand-checking we're comfortable with (so we go for high recall, low precision and check for false positives, etc.)?

Finally, should add that, with regard to dealing with missing data and difficult decision boundaries, I'm currently thinking about largely as a robustness issue: we keep track of it and see if it seems to be becoming a problem. That said, we can also treat it (a la Oliver et al. 2023) as itself an interesting set of empirical results.

## Where We Are Now

- DAPT on headlines/ledes: done
- trained on the CCA task: done, needs revision
  - Class prior estimation implemented using "Difference-of-Estimated-Densities-based Positive-Unlabeled Learning" (DEDPUL) (Ivanov 2020)
  - Focal loss implemented
  - Virtual adversarial training not implemented
- Other classification heads: not done

The quality of the current classifier: not great, but better than chance. Perhaps more importantly, I know how to improve it. A hand review of the least likely positives and most likely negatives indicates 1) the model has some idea what it's talking about; 2) I've been too generous with the NYT indexing service definitions. Fixing the second point is the highest leverage point of improvement.

## Where We're Going

### Next steps

- data stuff
  - make better use of DoCA by using the NYT API to pull down those 35 years of headlines + leads, then match DoCA into it, allowing us to use all the DoCA events as positives (instead of just those we are able to match into the NYT Annotated corpus; that is, at most, those between 1990 and 1995, I think).
  - review the quality of the LDC index labeled events (both with regard to CCA and immigration, ideally). there are definitely a bunch that made it in that just aren't very good (i.e., aren't actually cca events) and I need to 1) figure out why; and, 2) remove them (and any that are likely to be bad inclusions in the same way)
  - narrow down and hammer out our definition of CCA (potentially into multiple classes?). Are we trying to emulate DoCA as closely as possible? are we including lawsuits? boycotts? press conferences? etc. do we want to take a first pass with a narrower definition that will hopefully be less easily confused (i.e., focusing exclusively on “street” type events, or something similar).
    - \* this relates to the previous point about reviewing the LDC index events: it's quite possible that part of the issue is insufficient rigor or precision in my choices of which index descriptors to count for labeling purposes.
  - consider restricting the classifier to US events from the start? alternatively, make that a separate classification head as a first, easy go at a multi-head classifier.
- modeling stuff:
  - within this model:
    - \* implement Virtual Adversarial Training
    - \* hyperparameter search/tuning; learning parameters (rate, weight decay, scheduler, dropout), focal loss parameters, sensitivity sweep across class prior estimates,
    - \* implement per-layer learning rates, freezing/unfreezing, etc.
    - \* calibrate the outputs; can do this a number of ways, but don't want to just use a .5 threshold, for a number of reasons

- broadening from the CCA ID model: the ultimate goal is a model that will accurately ID *immigrant* contentious collective action, specifically. we want to do this by creating a multi-headed classifier; a shared encoder feeding into separate heads with different objectives; one classifying CCA/not, one immigrant involved/not, one trying to capture ICA overall, being fed partly by the other two, etc.
  - \* also worth testing just using multiple models (or even ensembling a single head multiples together with a multi-headed model?); not sure it'd be worth it though
  - \* so adding these heads, testing a few different structures and training procedures (i.e., are we still using VAT? if so, are we implementing that on a per-head basis? that dramatically increases training time, but might be worth it for generalizability and robustness, perhaps especially because of how individual the individual heads then feed into a combined output head); hyperparameter tuning

potentially adding some active learning elements: hard-negative mining (making sure to over-represent known difficult cases in training, e.g., make sure each batch has at least 20% drawn from the top 5k unlabeled by predicted p, or something); this could include a certain amount of hand classification of some of these cases (so we wind up with a couple hundred to a thousand difficult cases with known labels applied).

currently, model evaluation is somewhat hamstrung by the nature of the PU learning problem. we *can* evaluate the models, but we're definitely guessing to a certain degree. the solution to this is to hand classify a small subset of articles (as few as a couple hundred) drawn totally at random, so that we can then evaluate the performance of the model (trained on a PU dataset) on the actual PN problem that we care about. (this would also give another way of estimating the class prior)

this is a different thing than the active learning, hard-negative mining stuff I mention first; they're doing different things. the random draw is good for model evaluation because it gives us a set that is representative of the actual distribution we're training on so we can accurately evaluate performance with a minimum of re-weighting, calibration, etc. Hard-negative mining doesn't actually require any labeling at all, though it is more powerful if you do. Either way, it is a strategy for augmenting the training of the model, ensuring that it is actually getting a chance to learn from informative examples. If we do label cases, then we're doing even better: for the most informative cases, we've turned the PU problem into a PN problem (we can do this on a case-by-case basis by using a different loss and loss weighting technique for those cases).

## Down the Road

Some miscellaneous thoughts on what comes after the haystack problem. In other words, how to get a model that can usefully code the articles the first model identifies.

training the cca classifier to ID relevant sections of text? or, alternatively, restructuring the model so that we can tell what sections of the text are influencing the CCA/not judgement as a kind of workaround to get at a similar idea.

- use DoCA:
  - maybe this: we use the DoCA coding to train classifiers to ID the various pieces of info in article texts, then use that (maybe plus the overall CCA/not classifier) to tag relevant text by IDing the parts of the text that are influencing their decisions. then, use that labeling to train a “relevant info” classifier directly, which could then be used to classify sentences within an article.
  - could also potentially enforce a narrowing process? like, structure the model so that an early step (maybe within the transformer? or something) is explicitly about tossing out non-relevant parts of the text. then, once we’ve trained a bunch of classifiers on the different relevant pieces of info in this way, we take that first step and see how it does IDing the generally relevant-to-event text, or something?
  - alternatively (or also? this is maybe a different question), try and use DoCA to train an “event recognizer” to parse articles to ID and differentiate the event(s) reported within an article. something like the following: we train a set of models to be good at just pulling out the various pieces of info we care about from (small amounts of) text; e.g., a model to look at an article (or sentence?) and pull out participant numbers, targets, claims, whether there was violence, etc. Then, we use those as, essentially, post-processors to allow us to train an “article parser” where we give it an article and its task is to bucket the text from that article into not-event-related and event 1/event 2/etc. related. We then feed each of the “event” buckets into the “post-processors” and they spit out a set of event records, based on the text they were given. Then, the model is assessed based on whether the output records match any of the records in DoCA for that article.
    - \* this is probably too cute/over-complicated, but there’s maybe something here.

## References

- Bekker, Jessa, and Jesse Davis. 2020. “Learning from Positive and Unlabeled Data: A Survey.” *Machine Learning* 109(4):719–60. doi:[10.1007/s10994-020-05877-5](https://doi.org/10.1007/s10994-020-05877-5).
- Chollet, Francois. 2025. *Deep Learning with Python, Third Edition*. 3rd ed. New York: Manning Publications Co. LLC.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding.” Pp. 4171–86 in *Proceedings of the 2019 Conference of the North*. Minneapolis, Minnesota: Association for Computational Linguistics.
- Dhingra, Bhuwan, Jeremy R. Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, and William W. Cohen. 2022. “Time-Aware Language Models as Temporal Knowl-

- edge Bases.” *Transactions of the Association for Computational Linguistics* 10:257–73. doi:[10.1162/tacl\\_a\\_00459](https://doi.org/10.1162/tacl_a_00459).
- du Plessis, Marthinus C., Gang Niu, and Masashi Sugiyama. 2014. “[Analysis of Learning from Positive and Unlabeled Data](#).” in *Advances in neural information processing systems*. Vol. 27, edited by Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Q. Weinberger. Curran Associates, Inc.
- Earl, Jennifer, Andrew Martin, John D. McCarthy, and Sarah A. Soule. 2004. “The Use of Newspaper Data in the Study of Collective Action.” *Annual Review of Sociology* 65–80. doi:[10.1146/annurev.soc.30.012703.110603](https://doi.org/10.1146/annurev.soc.30.012703.110603).
- Elkan, Charles, and Keith Noto. 2008. “[Learning Classifiers from Only Positive and Unlabeled Data](#).” Pp. 213–20 in *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*. Las Vegas Nevada USA: ACM.
- Garg, Saurabh, Yifan Wu, Alexander J. Smola, Sivaraman Balakrishnan, and Zachary Lipton. 2021. “[Mixture Proportion Estimation and PU Learning: A Modern Approach](#).” Pp. 8532–44 in *Advances in neural information processing systems*. Vol. 34, edited by M. Ranzato, A. Beygelzimer, Y. Dauphin, P. S. Liang, and J. W. Vaughan. Curran Associates, Inc.
- Grimmer, Justin, and Brandon M. Stewart. 2013. “Text as Data: The Promise and Pitfalls of Automatic Content Analysis Methods for Political Texts.” *Political Analysis* 21(3):267–97. doi:[10.1093/pan/mps028](https://doi.org/10.1093/pan/mps028).
- Grootendorst, Maarten. 2022. “[BERTopic: Neural Topic Modeling with a Class-Based TF-IDF Procedure](#).”
- Gururangan, Suchin, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. 2020. “[Don’t Stop Pretraining: Adapt Language Models to Domains and Tasks](#).”
- Hanna, Alex. 2017. “MPEDS : Automating the Generation of Protest Event Data.” 1–40.
- Hutter, Swen. 2014. “Protest Event Analysis and Its Offspring.” *Methodological Practices in Social Movement Research* 335–67. doi:[10.1093/acprof:oso/9780198719571.003.0014](https://doi.org/10.1093/acprof:oso/9780198719571.003.0014).
- Ivanov, Dmitry. 2020. “[DEDPUL: Difference-of-Estimated-Densities-Based Positive-Unlabeled Learning](#).” Pp. 782–90 in *2020 19th IEEE International Conference on Machine Learning and Applications (ICMLA)*. Miami, FL, USA: IEEE.
- Ji, Zhanlin, Chengyuan Du, Jiawen Jiang, Li Zhao, Haiyang Zhang, and Ivan Ganchev. 2023. “Improving Non-Negative Positive-Unlabeled Learning for News Headline Classification.” *IEEE Access* 11:40192–203. doi:[10.1109/ACCESS.2023.3269304](https://doi.org/10.1109/ACCESS.2023.3269304).
- Kiryo, Ryuichi, Gang Niu, Marthinus C. du Plessis, and Masashi Sugiyama. 2017. “Positive-Unlabeled Learning with Non-Negative Risk Estimator.” Pp. 1674–84 in *Proceedings of the 31st international conference on neural information processing systems, NIPS’17*. Long Beach, California, USA: Curran Associates Inc.
- Lin, Tsung-Yi, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2020. “Focal Loss for Dense Object Detection.” *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42(2):318–27. doi:[10.1109/TPAMI.2018.2858826](https://doi.org/10.1109/TPAMI.2018.2858826).
- Liu, Xiaodong, Hao Cheng, Pengcheng He, Weizhu Chen, Yu Wang, Hoifung Poon, and Jianfeng Gao. 2020. “[Adversarial Training for Large Neural Language Models](#).”
- Liu, Yinhan, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy,

- Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. “[RoBERTa: A Robustly Optimized BERT Pretraining Approach](#).”
- McAdam, Doug, and Hillary Schaffer Boudet. 2012. *Putting Social Movements in Their Place: Explaining Opposition to Energy Projects in the United State, 2000-2005*. Cambridge: Cambridge University Press.
- Mikolov, Tomas, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. 2013. “[Distributed Representations of Words and Phrases and Their Compositionality](#).” in *Advances in neural information processing systems*. Vol. 26, edited by C. J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Q. Weinberger. Curran Associates, Inc.
- Miyato, Takeru, Shin-Ichi Maeda, Masanori Koyama, and Shin Ishii. 2019. “Virtual Adversarial Training: A Regularization Method for Supervised and Semi-Supervised Learning.” *IEEE Transactions on Pattern Analysis and Machine Intelligence* 41(8):1979–93. doi:[10.1109/TPAMI.2018.2858821](#).
- Oliver, Pamela, Alex Hanna, and Chaeyoon Lim. 2023. “Constructing Relational And Verifiable Protest Event Data: Four Challenges And Some Solutions.” *Mobilization: An International Quarterly* 28(1):1–22. doi:[10.17813/1086-671X-28-1-1](#).
- Oliver, Pamela, Chaeyoon Lim, Morgan Matthews, and Alex Hanna. 2022. “Black Protests in the United States, 1994 to 2010.” *Sociological Science* 9:275–312. doi:[10.15195/v9.a12](#).
- Phang, Jason, Thibault Févry, and Samuel R. Bowman. 2019. “[Sentence Encoders on STILTs: Supplementary Training on Intermediate Labeled-Data Tasks](#).”
- Pruksachatkun, Yada, Jason Phang, Haokun Liu, Phu Mon Htut, Xiaoyi Zhang, Richard Yuanzhe Pang, Clara Vania, Katharina Kann, and Samuel R. Bowman. 2020. “[Intermediate-Task Transfer Learning with Pretrained Language Models: When and Why Does It Work?](#)” Pp. 5231–47 in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics.
- Rosin, Guy D., Ido Guy, and Kira Radinsky. 2022. “[Time Masking for Temporal Language Models](#).” Pp. 833–41 in *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. Virtual Event AZ USA: ACM.
- Rosin, Guy D., and Kira Radinsky. 2022. “[Temporal Attention for Language Models](#).” Pp. 1498–508 in *Findings of the Association for Computational Linguistics: NAACL 2022*. Seattle, United States: Association for Computational Linguistics.
- Stoltz, Dustin S., and Marshall A. Taylor. 2019. “Concept Mover’s Distance: Measuring Concept Engagement via Word Embeddings in Texts.” *Journal of Computational Social Science* 2(2):293–313. doi:[10.1007/s42001-019-00048-6](#).
- Stoltz, Dustin S., and Marshall A. Taylor. 2021. “Cultural Cartography with Word Embeddings.” *Poetics* 88:101567. doi:[10.1016/j.poetic.2021.101567](#).
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. “[Attention Is All You Need](#).” in *Advances in neural information processing systems*. Vol. 30, edited by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Curran Associates, Inc.
- Voss, Kim, Steven Lauterwasser, and Irene Bloemraad. 2024. “Inactive and Quiescent? Im-

- migrant Collective Action in Comparative Perspective, 1960 to 1995.” *Socius: Sociological Research for a Dynamic World* 10. doi:[10.1177/23780231241249932](https://doi.org/10.1177/23780231241249932).
- Vu, Tu, Tong Wang, Tsendsuren Munkhdalai, Alessandro Sordoni, Adam Trischler, Andrew Mattarella-Micke, Subhransu Maji, and Mohit Iyyer. 2020. “[Exploring and Predicting Transferability Across NLP Tasks](#).” Pp. 7882–7926 in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics.
- Wankmüller, Sandra. 2022. “[Introduction to Neural Transfer Learning with Transformers for Social Science Text Analysis](#).”